

Databases

LABORATORY WORK 1

Student: Amanaly Zhaniya

Id: 25B031118

Part 1:

Task 1.1: Superkey and Candidate Key Analysis

Given the following relations, identify all superkeys and candidate keys:

Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

Sample Data:

EmpID	SSN	Email	Phone	Name	Department	Salary
101	123-45-6789	john@company.com	555-0101	John	IT	75000
102	987-65-4321	mary@company.com	555-0102	Mary	HR	68000
103	456-78-9123	bob@company.com	555-0103	Bob	IT	72000

Your Tasks: 1. List at least 6 different superkeys 2. Identify all candidate keys 3. Which candidate key would you choose as primary key and why? 4. Can two employees have the same phone number? Justify your answer based on the data shown.

Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Business Rules:

- A student can take the same course in different semesters
- A student cannot register for the same course section in the same semester
- Each course section in a semester has a fixed credit value

Relation A:

1.Superkeys: {EmpID}, {SSN}, {Email}, {EmpID, Name}, {EmpID, SSN}, {SSN, Department}

2.Candidate keys: {EmpID}, {SSN}, {Email}

Phone and Name aren't candidate keys — two employees could have the same name, and phone numbers can be shared (e.g. office line).

3.Primary key: EmpID. It's short, numeric, doesn't change, and unlike SSN or Email it's not sensitive personal info.

4.Can two employees share a phone number?

Yes — nothing stops two employees from sharing an office phone, so Phone isn't reliable as a key even though it looks unique in this sample.

Relation B:

1.Primary key: {StudentID, CourseCode, Section, Semester, Year}

2.Why each part is needed:

- StudentID – which student
- CourseCode – which course
- Section – multiple sections of the same course can run in one semester
- Semester, Year – a student can retake the same course later, so these separate different attempts

If any of these were dropped, two different registrations could end up looking identical.

3.Other candidate keys: none — Grade and Credits just describe the registration, they don't identify it.

Task 1.2: Foreign Key Design

Design the foreign key relationships for this university system:

Given Tables:

Student(StudentID, Name, Email, Major, AdvisorID)

Professor(ProfID, Name, Department, Salary)

Course(CourseID, Title, Credits, DepartmentCode)

Department(DeptCode, DeptName, Budget, ChairID)

Enrollment(StudentID, CourseID, Semester, Grade)

Your Tasks: 1. Identify all foreign key relationships

Foreign keys:

- Student.AdvisorID → Professor.ProfID
- Course.DepartmentCode → Department.DeptCode
- Department.ChairID → Professor.ProfID

- Enrollment.StudentID → Student.StudentID
- Enrollment.CourseID → Course.CourseID

Part 2:

1) Strong – Patients, Doctors, Departments, Appointments, Prescriptions **Weak** – Hospital Rooms

2) Patients :

- PatientID – Primary key, simple
- Name – simple
- Birthdate – simple
- Address – composite: (street,city,state,zip)
- PhoneNumber – multi-valued
- Insurance – simple

Departments:

- DepartmentCode – Primary key
- Name – simple
- Locations – simple
- Purpose – simple
- Notes – simple

Hospital Rooms:

- RoomNumber - partial key
- DepartmentCode – Foreign key

Doctors:

- DoctorID – Primary key, simple
- Name – simple
- Specialization – multi-valued
- PhoneNumber - simple
- OffLocation – simple

Appointments:

- AppID – Primary key
- DoctorName – Simple
- Date - Simple

Prescriptions:

- PrescriptionsID – Primary key
- Medications – simple
- Patients – simple
- Dosage – simple

3) Relationships with cardinalities:

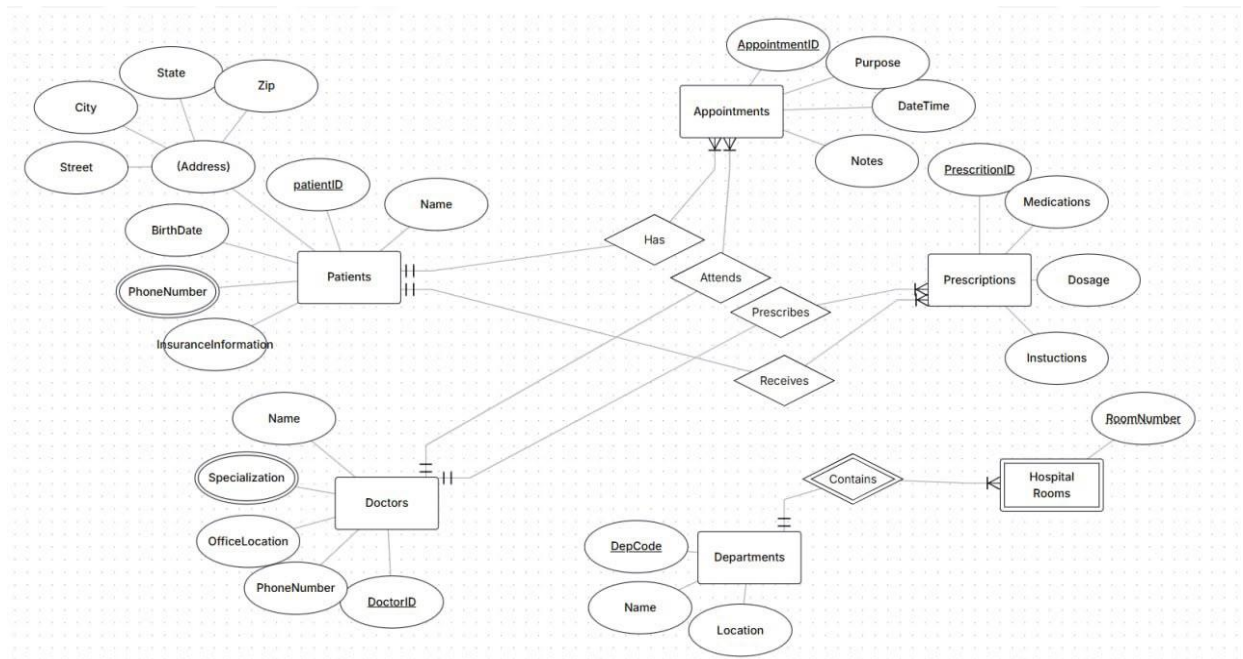
1 : 1 – we dont have this type relations

1 : N – Patient->Appointment, Doctor->Appointment, Doctor-> Prescriptions

Patient-> Prescriptions, Department->HospitalRooms

Patient N:M Doctor (with Appointment)

4)



5) Pk are marked with underline

Task 2.2: E-commerce Platform

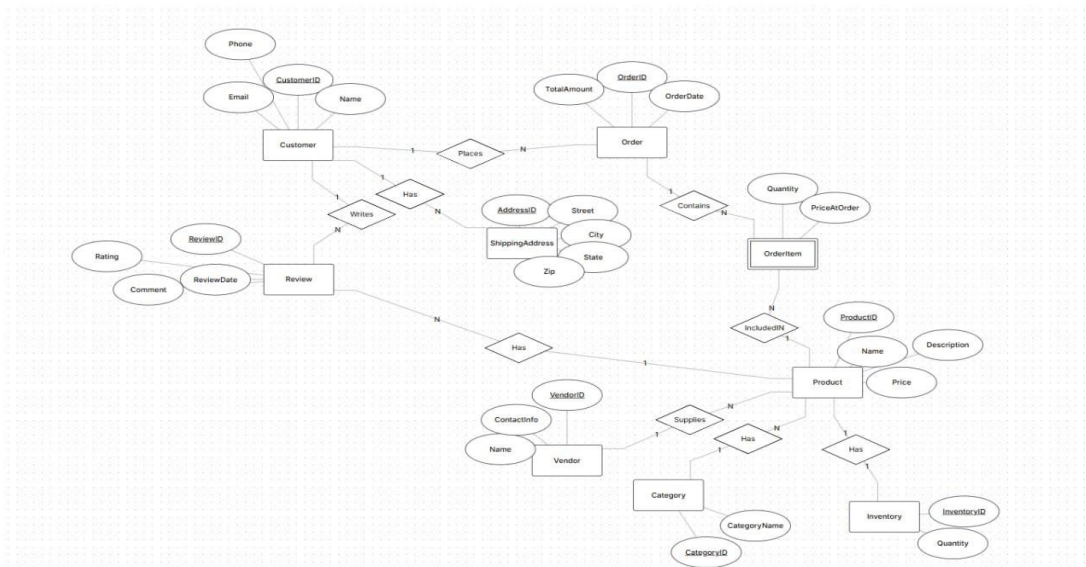
Scenario: Design a simplified e-commerce database.

Requirements:

- **Customers** place **Orders** for **Products**
- **Products** belong to **Categories** and are supplied by **Vendors**
- **Orders** contain multiple **Order Items** (quantity and price at time of order)
- **Products** have reviews and ratings from customers
- Track **Inventory** levels for each product
- **Shipping addresses** can be different from customer billing addresses

Your Tasks: 1. Create a complete ER diagram 2. Identify at least one weak entity and justify why it's weak 3. Identify at least one many-to-many relationship that needs attributes

1)



2) OrderItem is a weak entity because it cannot exist without an Order and a Product

3) Order and Product have a many-to-many relationship through OrderItem. The relationship has attributes Quantity and PriceAtOrder.

Part 4:

Task 4.1: Denormalized Table Analysis

Given Table:

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

Your Tasks: 1. **Identify functional dependencies:** List all FDs in the format $A \rightarrow B$ 2. **Identify problems:** - What redundancy exists in this table? - Give specific examples of update, insert, and delete anomalies 3. **Apply 1NF:** Are there any 1NF violations? How would you fix them? 4. **Apply 2NF:** - What is the primary key of this table? - Identify any partial dependencies - Show the 2NF decomposition 5. **Apply 3NF:** - Identify any transitive dependencies - Show the final 3NF decomposition with all table schemas

- 1) StudentID $\rightarrow$ StudentName, StudentMajor
 ProjectID $\rightarrow$ ProjectTitle, ProjectType
 SupervisorID $\rightarrow$ SupervisorName, SupervisorDept
 StudentProject (StudentID, ProjectID) $\rightarrow$ Role, HoursWorked, StartDate, EndDate

2) redundancy:

- Name and major of student are repeated for each project
- The name and department of the supervisor are repeated for each student working on the project.

Anomalies:

Update: If the head's department changes, all lines where it appears need to be updated.

Insert: You cannot add a new student without a project, as the ProjectID is mandatory.

Delete: If you delete the last entry about the project, you will lose information about the project itself and its leader.

3) The table is already in 1NF. (all values are atomic)

4) Primary Key: (StudentID, ProjectID)

Partial dependences:

StudentID -> StudentName, StudentMajor

ProjectID -> ProjectTitle, ProjectType

SupervisorID -> SupervisorName, SupervisorDept

2nf: Student (StudentID, StudentName, StudentMajor)

Project (ProjectID, ProjectTitle, ProjectType, SupervisorID)

Supervisor (SupervisorID, SupervisorName, SupervisorDept)

StudentProject (StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5) transitive dependences: Project: SupervisorID-> SupervisorName, SupervisorDept (with SupervisorID)

3nf: Student (StudentID, StudentName, StudentMajor)

Project (ProjectID, ProjectTitle, ProjectType, SupervisorID)

Supervisor (SupervisorID, SupervisorName, SupervisorDept)

StudentProject (StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

Task 4.2: Advanced Normalization

Given Table:

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName,
InstructorID, InstructorName, TimeSlot, Room, Building)

Business Rules:

- Each student has exactly one major
- Each course has a fixed name
- Each instructor has exactly one name
- Each time slot in a room determines the building (rooms are unique across campus)
- Each course section is taught by one instructor at one time in one room
- A student can be enrolled in multiple course sections

Your Tasks: 1. Determine the primary key of this table (hint: this is tricky!) 2. List all functional dependencies 3. Check if the table is in BCNF 4. If not in BCNF, decompose it to BCNF showing your work 5. Explain any potential loss of information in your decomposition

1) Primary Key (StudentID, CourseID, TimeSlot, Room)

2) from Business Rules:

StudentID → StudentMajor CourseID → CourseName

InstructorID → InstructorName Room → Building

(CourseID, TimeSlot, Room) → InstructorID

(StudentID, CourseID, TimeSlot, Room) → all other attributes (this is the key).

3) BCNF — every determinant must be a superkey

StudentID → StudentMajor (StudentID is not a key).

CourseID → CourseName (CourseID is not a key).

InstructorID → InstructorName (InstructorID is not a key).

Room → Building (Room is not a key).

Table is not BCNF

4) Decompose to BCNF

Students(StudentID, StudentMajor)

Courses(CourseID, CourseName)

Instructors(InstructorID, InstructorName)

Rooms(Room, Building)

CourseSections(CourseID, TimeSlot, Room, InstructorID)

Enrollments(StudentID, CourseID, TimeSlot, Room)

5) There is no data loss, but there is a need to join tables via JOIN to obtain the original information.

For example, to find out a student's schedule, you need to join Enrollments with CourseSections, and then with Courses, Instructors, and Rooms.

Part 5:

Task 5.1: Real-World Application

Scenario: Your university wants to track student clubs and organizations with the following requirements:

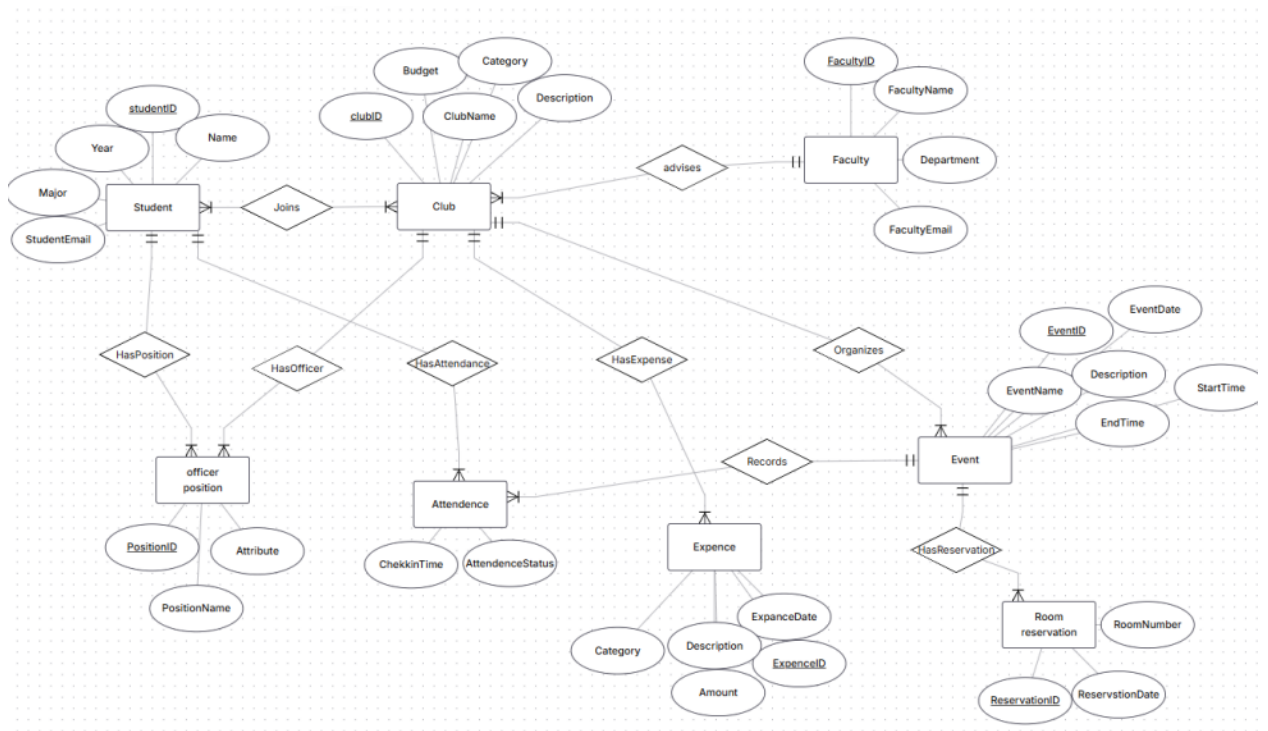
System Requirements:

- Student clubs and organizations information
- Club membership (students can join multiple clubs, clubs have multiple members)
- Club events and student attendance tracking
- Club officer positions (president, treasurer, secretary, etc.)
- Faculty advisors for clubs (each club has one advisor, faculty can advise multiple clubs)
- Room reservations for club events
- Club budget and expense tracking

Your Tasks: 1. Create a complete ER diagram for this system 2. Convert your ER diagram to a normalized relational schema 3. Identify at least one design decision where you had multiple valid options and explain your choice 4. Write 3 example queries that your database should support (in English, not SQL)

Example Query Format: - "Find all students who are officers in the Computer Science Club" - "List all events scheduled for next week with their room reservations"

1)



2)

Relational schema:

Student(StudentID, Name, Email, Major)

FacultyAdvisor(AdvisorID, Name, Department)

Club(ClubID, ClubName, AdvisorID*, Budget)

Membership(StudentID*, ClubID*, JoinDate, OfficerRole)

Event(EventID, ClubID*, EventName, DateTime, RoomID*)

Room(RoomID, Building, Capacity)

Attendance(StudentID*, EventID*, Status)

Expense(ExpenseID, ClubID*, Amount, Date, Description)

3)

- **Store OfficerRole as a simple attribute in the Membership table** — simpler, assumes a student holds only one role in a club at a given time.
- **Create a separate OfficerPosition table** (StudentID, ClubID, Position, StartDate, EndDate) — allows tracking the history of officer roles over time (e.g. if someone held different positions across semesters).

4)

1. "Find all students who are officers in the Computer Science Club"
2. "List all events scheduled for next week with their room reservations"
3. "Show the total expenses for each club this year, grouped by club"

